

AMAS Write-up: Hidden Markov Models

Authors: R. Ahrensberg, S. Clemensen, A. Roelshøj, and A. G. Wolstrup

Date: 4/3-2026

1 Introduction

A Hidden Markov Model (HMM) is an augmentation of a Markov chain. A Markov chain is useful when computing the probability for a sequence of observable events, where the next step depends only on the current state. In many realistic cases, however, the events of interest are not observable but hidden. To address this problem, HMMs are used, since they make it possible to infer the probability of the underlying sequence of hidden events based solely on the observed events.

2 Hidden Markov Models

In an HMM, there are two linked processes consisting of a hidden sequence of states that evolves according to transition probabilities, and a sequence of observations generated from those states according to emission probabilities [1].

Formally, HMMs consist of an initial state sequence π , a transition matrix A , and an emission matrix B , and are denoted $\lambda = (\pi, A, B)$. The transition matrix describes the probabilities associated with transitions between hidden states (green arrows on Fig. 1). The emission matrix describes probabilities of each observation given each hidden state (red arrows on Fig. 1) [1].

To understand this concept, we'll explore a simplified example. A teacher wants to know whether their students are spending time at home preparing for class. Because the students refuse to complete the course evaluation, the teacher cannot directly observe their study habits. Instead, the teacher records whether each student finishes the in-class exercise on time. The goal is to use these observations to infer whether a given student came to class prepared or unprepared.

We assume there are only two possible hidden states: Prepared (P) and Not Prepared (N), and two possible observations: Finished on Time (F) and Did Not Finish on Time (D). The task, then, is as follows: Given a sequence of observations O (each either F or D, representing whether the student completed the exercise on time), determine the most likely sequence of hidden states Q (P or N) that explains the student's behavior.

In order to compute this sequence, the model

includes probabilities that describe how likely a prepared or unprepared student is to finish on time, as well as probabilities that capture how likely a student is to remain prepared or unprepared from one class session to the next or change behavior. This example is showcased in Fig. 1, where green denotes the hidden states and transition probabilities, while red denotes the observable states and emission probabilities.

In real applications, HMMs have proven especially useful for speech recognition [1]. Such a model can be trained by providing a vocabulary of words along with a set of examples of the words being spoken. Thus the speech data is the observation, and the actual word or sentence is the hidden state to be determined. Similar models can be constructed for numerous other applications.

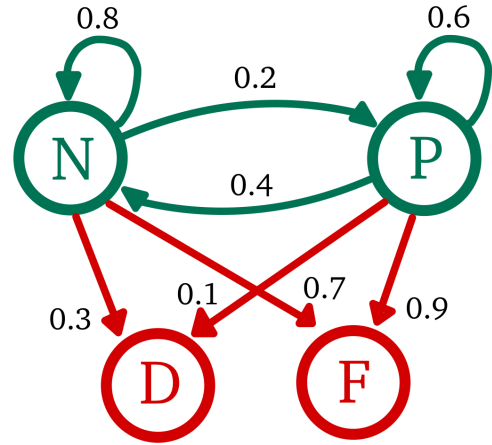


Figure 1: An HMM for studying whether students spend time preparing for class (P) or not (N) based on whether they finish exercises on time (F) or not (D). Green denotes the hidden states and transition probabilities, while red denotes the observable states and emission probabilities.

3 Algorithms

For HMMs, the main problem to solve is: Given a sequence of observations, O , determine the sequence of hidden states, Q . This problem is condensed into three problems [1]:

1. (Likelihood) Given an HMM, $\lambda = (\pi, A, B)$, and an observation sequence O , what is the

probability that O was produced by λ ?

2. (Decoding) Given an observation sequence, O , and an HMM, $\lambda = (\pi, A, B)$, what is the most probable hidden state sequence, Q ?
3. (Learning) Given an observation sequence, O , and the states of the HMM, learn the parameters π , A , and B .

In most use-cases, the problems are solved from 3) to 1).

There are three algorithms dedicated to solving the aforementioned problems, called the Forward-Backward algorithm (Likelihood), the Viterbi algorithm (Decoding) and the Baum-Welch algorithm (Learning).

The likelihood step is simply to calculate, given a λ and an observation sequence $O = o_1, o_2, \dots, o_T$, the likelihood $P(O|\lambda)$. So the task is for each possible observation sequence to calculate the associated probability within your HMM. The Forward-Backward algorithm stores intermediate values and finds the probability for each state by adding up the different paths that lead to the given state. Thus, the probability at step t is

$$\alpha_t(j) = \sum_{i=1}^N \alpha_{t-1}(i) a_{ij} b_j(o_t), \quad (1)$$

where a_{ij} is the transition probability from state i to state j , and $b_j(o_t)$ is the probability of observation o_t given state j [1].

Similarly to the Forward-Backward algorithm, Decoding is done by storing intermediate values by the Viterbi algorithm. At step t , the value

$$v_t(j) = \max_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t) \quad (2)$$

is computed. At the final step, $t = T$, this will result in the most likely sequence $Q = \max_{i=1}^N v_T(i)$, as well as the associated probability $P(Q \cup O)$, that is, the probability of having both the observation and the state sequence. Together with step 1, this produces the desired probability $P(Q|O) = P(Q \cup O)/P(O)$ [1].

Both of these algorithms assume that the matrices A and B are known. They are, however, typically not available in advance, which is why these algorithms are usually performed last.

The learning step is done using the Baum-Welch algorithm, which estimates A , B , and π , by adjusting the parameters to maximize the probability of the observation sequence given λ . It is an iterative algorithm, which computes probabilities

from the initial state, and repeats until the probabilities converge [1].

The central object we need for this computation is the probability of being in state i at time t and state j at time $t + 1$, given the observation sequence and the model:

$$\xi(i, j) = P(q_t = i, q_{t+1} = j | O, \lambda) \quad (3)$$

$$= \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{\sum_{j=1}^N \alpha_t(j) \beta_{t+1}(j)}. \quad (4)$$

Here, $\beta_t(j)$ is the backward probability. It is the probability of seeing a sequence of observations given that the system is in state j :

$$\beta_t(j) = P(o_{t+1}, o_{t+2}, \dots, o_T | q_t = j, \lambda). \quad (5)$$

It is found in much the same way as the forward probability (Eq. 1), but is done recursively backwards. The matrix elements a_{ij} of A and $b_j(o_t)$ are found in terms of the ξ 's and iteratively re-estimated, forming the core of the algorithm [1].

Although this should in principle work unperturbed, it is in practice very sensitive to the initial conditions. Therefore, the structure of the HMM is often given beforehand as with the speech recognition mentioned previously - e.g. the number of possible states or some other known behavior of the system of interest.

4 Conclusion

Hidden Markov models can be a powerful statistical tool for determining a hidden sequence of events from a sequence of observed events. Using the three algorithms and our data, we can go from a sequence of observable events to emission and transition probabilities (learning), then to determining the optimal hidden state sequence (decoding) and then compute how likely the hidden state sequence is given the observable sequence (likelihood). This relatively straightforward process makes HMMs an excellent statistical tool for predicting future hidden states, provided that the underlying data satisfy the model assumptions. It is no wonder that they are found in many areas of modern research and technology.

References

- [1] L. R. Rabiner et al. "An introduction to hidden Markov models". In: *IEEE ASSP Magazine* 3 (1986), pp. 4–16. URL: <https://api.semanticscholar.org/CorpusID:11358505>.